

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Петрозаводский государственный университет»

Институт математики и информационных технологий
кафедра прикладной математики и кибернетики

Анисимов Никита Николаевич

Алгоритмы управления запросами на тестирование для
оптимизации среднего и максимального времени ожидания
пользователя автоматической тестирующей системы

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Направление подготовки бакалавра
09.03.04 – Программная инженерия
Форма обучения - очная

Допущена к защите.
Заведующий кафедрой
к.ф.-м.н., доцент
И. В. Пешкова

Руководитель
д.т.н., профессор
В. А. Кузнецов

«_____» _____ 2024 г.

«_____» _____ 2024 г.

Оглавление

Введение	4
1 Предметная область	6
1.1 Определения	6
1.2 Аналогии	10
1.2.1 Ejudge	10
1.2.2 Fsystem	10
1.2.3 Яндекс.Контест	11
1.2.4 Codeforces	12
2 Математическая модель	13
2.1 Пуассоновский поток	13
2.1.1 Определение	13
2.1.2 Интуитивное наблюдение	14
2.1.3 Сбор и обработка реальных данных	14
2.1.4 Проверка гипотезы	15
2.2 Система массового обслуживания	16
2.2.1 Определения	16
2.2.2 Тип системы	17
2.3 Расчет оценочных значений	17
2.4 Специализация исполнителей	18
3 Алгоритм управления очередью	21
3.1 Целевая функция	21
3.2 Разработка алгоритмов	21
3.2.1 FifoQueue	21

3.2.2	OnlyAskQueue	22
3.2.3	MaxFirstQueue	22
3.2.4	TestByTestQueue	22
3.2.5	SortTestQueue	23
3.3	Имитационная модель	23
3.4	Анализ алгоритмов	27
3.4.1	OnlyAskQueue	27
3.4.2	MaxFirstQueue	28
3.4.3	TestByTestQueue	29
3.4.4	SortTestQueue	30
3.5	Компиляция	31
3.6	Итоговый алгоритм	31
4	Разработка программного модуля	33
4.1	Проектирование	33
4.2	Разработка	37
	Заключение	39
	Библиографический список использованной литературы	40

Введение

Соревнования по спортивному программированию в России признаны спортом и проводятся в следующих спортивных дисциплинах:

- программирование алгоритмическое;
- программирование продуктивное;
- программирование беспилотных авиационных систем;
- программирование робототехники;
- программирование систем информационной безопасности.

На соревнованиях по алгоритмическому программированию участники отправляют свои решения в автоматическую тестирующую систему, которая сообщает вердикты по их посылкам. Уменьшение времени ожидания обработки посылки является важной задачей для проведения соревнований по программированию.

В тестирующей системе возникает очередь посылок, откуда они отправляются на специальные сервера для тестирования. Один из подходов к уменьшению времени обработки посылок — это исследование и оптимизация очереди посылок.

Цель работы: проектирование и разработка модуля для управления очередью посылок на тестирование для соревнований по алгоритмическому программированию.

Задачи работы:

1. Составить математическую модель по предметной области;
2. Изучить аналоги тестирующих систем;
3. Разработать алгоритм управления очередью;

4. Спроектировать, разработать и внедрить программный модуль, реализующий алгоритм управления очередью.

Глава 1

Предметная область

1.1 Определения

Алгоритмическое программирование — методология написания программного кода, целью которой является написание оптимизированного по времени и памяти кода, который эффективно решает поставленные задачи за счет применения различных алгоритмов.

Контекст — соревнование по алгоритмическому программированию, где участникам предлагается некоторый набор задач, решениями к которым являются программы. Участники посылают решения по задачам в тестирующую систему, в которой они проходят ряд заранее подготовленных тестов. Тестирующая система сообщает участникам их вердикты, баллы и штраф, полученные за задачи.

Рассматриваются следующие форматы соревнований:

- ICPC style — задача считается сданной, если программа, посланная участником в тестирующую систему, прошла все тесты этой задачи. Тестирование продолжается до первого по номеру теста, на котором посылка участника отработает неправильно. Участнику сообщается о номере этого теста и вердикт, полученный при его тестировании. В турнирной таблице участники сортируются по количеству сданных задач, а при равенстве - по штрафу, который начисляется в числе 20 минут за каждую неверную попытку и также за каждую минуту контекста до первой сдачи задачи;
- IOI style — тесты разделены на пронумерованные группы, каж-

дая группа тестируется целиком. Чтобы посылка протестировалась на группе, необходимо, чтобы она успешно прошла определенные предыдущие по номеру группы. Участнику сообщаются вердикты на всех протестированных тестах, а также начисляются баллы за тесты, на которых посылка вывела правильный ответ и уложилась в ограничения. Время сдачи и количество попыток не играют роли. В турнирной таблице участники отсортированы по баллам.

Синонимы: соревнование, соревнование по программированию.

Турнирная таблица — список участников, динамически обновляющийся во время соревнования, и показывающий, какое место на текущий момент занимает каждый участник. Также в турнирной таблице содержится информация о штрафе, количестве и статусе посылок всех участников.

Синонимы: таблица.

Задача — основной предмет оценки спортивного программиста, общепринятый по всему миру.

Цифровую запись задачи можно разбить на:

1. Чекер — программа, проверяющая, что вывод посылки участника на данный тест является правильным;
2. Валидатор — программа, проверяющая, что тесты отвечают заданным ограничениям;
3. Условие — текст задачи, доступный участникам;
4. Главное решение — решение автора задачи, которое считается правильным;
5. Тесты — набор данных, которые подаются на ввод программе участника, чтобы проверить правильность решения задачи.
6. Интерактор — программа, которая взаимодействует с решением участника в задачах интерактивного вида;
7. Файл конфигурации.

Посылка участника — исходный код программы, который участник считает решением задачи. Для совершения посылки участник выбирает, по какой задаче он хочет ее сделать и какой компилятор необходимо использовать системе для компиляции его программы.

Синонимы: программа, программа участника, посылка, решение.

Тестирующая система — информационная система, реализующая тестирование программ участников по задачам в контекстах, обновление и хранение турнирных таблиц и историй посылок участников, Синонимы: система.

Тестирование — процесс запуска решения участника на ряде тестов и проверка правильности вывода.

Вердикт — сообщение о статусе протестированной программы.

- ОК — Программа прошла все тесты;
- WA x — Программа выдала неправильный вывод на тесте номер x;
- TL x — Программа превысила ограничение по времени на выполнение на тесте номер x;
- ML x — Программа превысила ограничение по использованию оперативной памяти на тесте номер x;
- RE x — Программа завершилась с ошибкой выполнения на тесте номер x;
- CE — Программа не скомпилировалась;
- PE x — Программа выдала вывод в неправильном формате на тесте номер x.

Для IOI style контекстов сообщаются вердикты для каждого протестированного теста.

Дорешивание — поток посылок в систему вне соревнований.

Жюри — группа людей, ответственная за создание пакета задач, отслеживание системы во время проведения соревнования и ответы на вопросы

Джадж — сервер, тестирующий решения участников.

Синонимы: исполнитель, judge.

Очередь посылок — очередь решений участников, отправленных на сервер, но еще не взятых на тестирование.

Перетестирование — процесс повторного тестирования решений участников на измененных тестах или с измененными ограничениями в рамках соревнования.

1.2 Аналоги

Существует множество аналогов тестирующих систем с различными подходами к тестированию. Здесь перечислены самые популярные из них и их отличительные особенности.

1.2.1 Ejudge

Ejudge — единственная тестирующая система с открытым исходным кодом и подробной документацией. Для управления посылками используется очередь с приоритетом, причем приоритет меняется вручную для задачи, языка программирования или контеста. Также задачи, отправленные на перетестирование, становятся более приоритетными.

Система поддерживает параллельное тестирование посылки в рамках одного исполнителя.

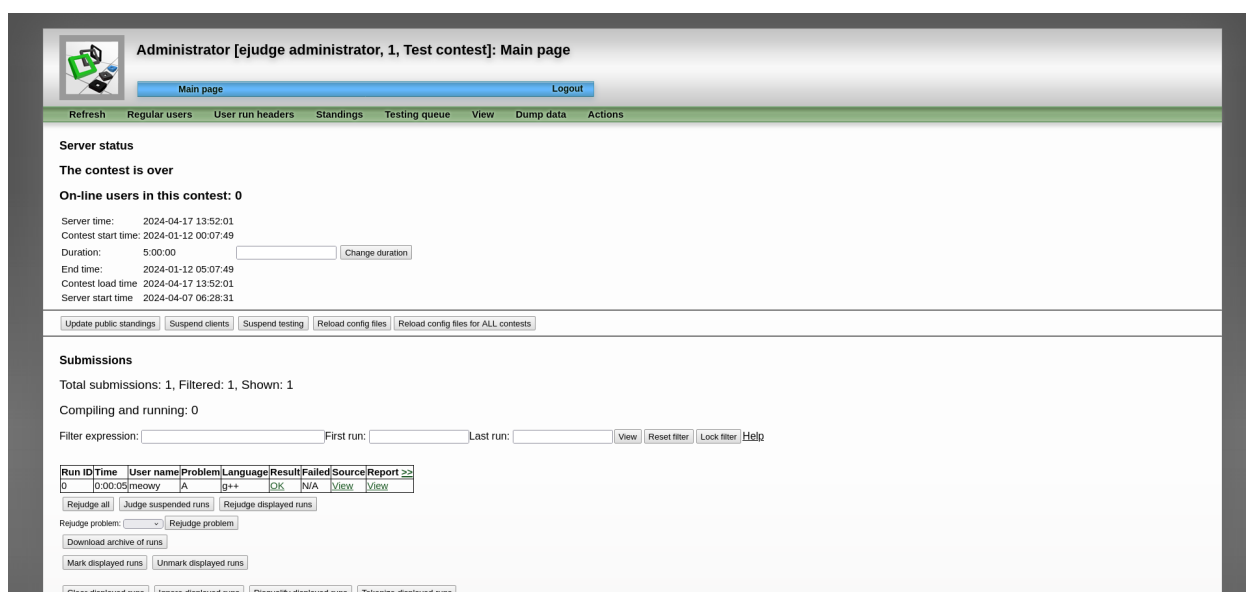


Рис. 1.1: Ejudge

1.2.2 Fsystem

Fsystem — устаревшая система, написанная в ПетрГУ и являющаяся базой для разрабатываемой тестирующей системы. У системы нет документации, но по лог-файлам можно предположить, что в ней используется потестовое распараллеливание при тестировании.



Информация о конкурсе

Отправить решение

Результаты тестирования

Положение команд

Сообщения

Распечатать решение

Выход

Информация о конкурсе:

Название конкурса:

XXIX Личный чемпионат ПетрГУ (весенний сезон), 28 мая 2023.

Время начала:

2023-05-28 10:00:00

Продолжительность:

300

Время окончания:

2023-05-28 15:00:00

Условия задач:

[Условия](#)

Задачи:

ID	Название	TimeLimit	MemoryLimit	Input	Output	Read	Send	Послано/Сдано
A	Рон Уизли и история магии	1 s	256 megabytes	stdin	stdout			31/27 (87.10%)
B	Опять задача про столы	1 s	256 megabytes	stdin	stdout			45/25 (55.56%)
C	Гермиона и Гоша	1 s	256 megabytes	stdin	stdout			36/25 (69.44%)
D	У Гарри день рождения!	1 s	256 megabytes	stdin	stdout			63/15 (23.81%)
E	Леша, магия и клубы	1 s	256 megabytes	stdin	stdout			35/23 (65.71%)
F	Гарри, Рон, Гермиона и Лодейное поле	1 s	256 megabytes	stdin	stdout			22/12 (54.55%)
G	Двадцать три проблемы Николаса Фламеля	1500 ms	256 megabytes	stdin	stdout			17/7 (41.18%)
H	Классики нового поколения	2 s	256 megabytes	stdin	stdout			28/0 (0.00%)
I	Школа Магии и Волшебства и ТППО	2500 ms	512 megabytes	stdin	stdout			33/1 (3.03%)
J	Письмо из хогвартса	4 s	256 megabytes	stdin	stdout			22/0 (0.00%)
K	Стажировка в Министерстве Магии	1500 ms	256 megabytes	stdin	stdout			0/0

Рис. 1.2: Fsystem

1.2.3 Яндекс.Контеcт

Яндекс.Контеcт — система от компании Яндекс. Несколько трудна в использовании, обладает неудобным интерфейсом. В большинстве систем во время тестирования пользователям сообщается о том, сколько тестов решение уже прошло. В Яндекс.Контеcт сообщается только о прохождении каждых 10 тестов.



Контеcт

Соревнования

Наборы задач

Задачи

В прошлую версию

?

➡

🔔

Список соревнований

RUCODE. CTAPT 2024. Division F.CPP

Монитор

Страница Enter

Посылки

Сообщения

Доступ и участники

Настройки

Общие

Посылки

Компиляторы

Тестирование и отчет для участников

Монитор

Интерфейс участника

Комментарии к решениям

Задачи

Скачать архив

Минимальное время между попытками, в секундах

1

Максимальное кол-во попыток в секунду: 1

Исключить вердикты

—

Посылки с идентичным кодом

☒

Компиляторы

Очистить

basic

☐ Free Basic 1.04

c

☐ GNU-GCC 7.3-G++

☐ GNU GCC 13.1 C17

c#

☐ C# (MS-.NET 5.0 + ASP)

☐ C# (MS-.NET 6.0 + ASP)

☐ Mono C# 5.2.0

kumir

☐ kumir 2.1.0-rc9

lua

☐ Lua 5.4

make

☐ (Make) Clang 17.0.1 C++20

☐ (Make)-C# (MS-.Net-Core 5.0)

☐ (Make) C# (MS-.Net-Core 6.0)

☐ (make) GNU GCC 13.1 C17

☐ (make) GNU GCC 13.1 C++20 + JSON

perl

☐ Perl 5.14

php

☐ PHP-7.3.5

☐ PHP 8.1

python

☐ Python 3.9 (PyPy 7.3.11)

☐ Python-2.7

☐ Python 3.12.1

Рис. 1.3: Яндекс.Контеcт

1.2.4 Codeforces

Codeforces — самая популярная платформа по спортивному программированию в России. Отличительной особенностью системы Codeforces является проведение на ней раундов — онлайн соревнований, в которых участвует большое количество людей. В целях оптимизации времени тестирования на раундах для задачи подготавливается небольшое (в среднем от 2 до 10) количество претестов, на которых решения тестируются во время соревнования. Также во время соревнования у участников есть возможность «взломать» послыки других участников, подобрав тест, на которых их решения выдают ответ, отличный от правильного. После раунда начинается системное тестирование, в рамках которого решения проходят дополнительную проверку на добавленных участниками и жюри тестах.



Sponsored by TON

🇷🇺 | 🇬🇧 | 🇺🇸

[alyoks1](#) | [Выйти](#)

ГЛАВНАЯТОПКАТАЛОГСОРЕВНОВАНИЯТРЕНИРОВКИАРХИВГРУППЫРЕЙТИНГEDUAPIКАЛЕНДАРЬПОМОЩЬ

ЗАДАЧИОТosлатьМОИ ПОСЫЛКИСТАТУСПОЛОЖЕНИЕЗАПУСК

Задачи

№	Название			
A	Chinese Girls' Amusement¹	china.in / china.out 1 c, 64 MB		x667
B	Reactor Cooling¹	cooling.in / cooling.out 1 c, 64 MB		x409
C	New Year Bonus Grant¹	grant.in / grant.out 1 c, 64 MB		x491
D	Matrix Multiplication¹	matrix.in / matrix.out 1 c, 64 MB		x601
E	Nice Patterns Strike Back¹	nice.in / nice.out 1 c, 64 MB		x479
F	Get Out!¹	out.in / out.out 1 c, 64 MB		x170
G	Beautiful People¹	people.in / people.out 1 c, 64 MB		x371
H	Cracking' RSA¹	rsa.in / rsa.out 1 c, 64 MB		x279

Контеcты [Андрея Станкевича](#)

Открытая

Участник

→ Соревнования группы

- 2014-2015 Зимние Петрозаводские сборы, Andrew Stankevich Contest 47 (ASC 47)
- 2014-2015 Летние Петрозаводские сборы, Andrew Stankevich Contest 46 (ASC 46)
- 2013-2014 Зимние Петрозаводские сборы, Andrew Stankevich Contest 45 (ASC 45)
- 2013-2014 Летние Петрозаводские сборы, Andrew Stankevich Contest 44 (ASC 44)

¹ Пакет к этой задаче не обновлялся авторами или администрацией Codeforces после смены тестирующих серверов проекта на более быстрые. По этой причине, чтобы сохранить актуальность ограничения по времени, время исполнения решений умножается на два. Например, если ваше решение отработало на сервере за 400 мс, то для определения вердикта и отображения на сайте будет использовано значение 800 мс.

[Все задачи](#)

Рис. 1.4: Codeforces

Глава 2

Математическая модель

2.1 Пуассоновский поток

2.1.1 Определение

Пуассоновский поток — случайный процесс, который описывает количество наступивших случайных событий, происходящих с постоянной интенсивностью. Число событий на отрезке времени в таком процессе подчинено распределению Пуассона. Пусть t — длина произвольного промежутка времени, λ — интенсивность событий (количество в единицу времени). Тогда вероятность количества событий m на этом промежутке можно вычислить по формуле 2.1.

$$P_t(m) = \frac{(\lambda t)^m}{m!} e^{-\lambda t} \quad (2.1)$$

Интервалы между событиями распределены по экспоненциальному распределению с параметром λ . Плотность экспоненциального распределения можно вычислить по формуле 2.2.

$$f(x) = \lambda e^{-\lambda x} \quad (2.2)$$

Пуассоновский поток часто используется для моделирования запросов к серверу. Критериями для определения процесса как пуассоновского служат:

1. Стационарность — вероятностные характеристики процесса не зависят от времени;

2. Отсутствие последствия — происходящее до момента времени t не влияет на характеристики потока после момента t , для любого t ;
3. Ординарность — вероятность появления двух или более событий на весьма малом участке времени пренебрежительно мала.

2.1.2 Интуитивное наблюдение

Опыт участия в соревнованиях показывает, что поток посылок на контесте не является стационарным. В первые минуты совершается большое количество посылок на простые задачи, которые можно быстро придумать. В середине контеста интенсивность посылок уменьшается, на последние часы зачастую остаются самые сложные для участников задачи — интенсивность становится еще ниже. Однако, если разбить контест на блоки начало-середина-конец и исследовать их независимо, можно предположить, что стационарность внутри блоков будет выполняться.

2.1.3 Сбор и обработка реальных данных

В качестве реальных данных для проверки гипотезы о том, что поток посылок в определенный отрезок времени контеста — Пуассоновский, использовались данные со следующих соревнований:

- Зимний чемпионат ПетрГУ 2023 года;
- Весенний чемпионат ПетрГУ 2019 года;
- Школьный этап Всероссийской олимпиады школьников 2023 года, проводимый в Петрозаводске;
- Школьный этап Всероссийской олимпиады школьников 2022 года, проводимый в Петрозаводске.

Все выбранные соревнования проводились на системе Fsystem, в которой было сохранено время каждой посылки.

2.1.4 Проверка гипотезы

Для проверки гипотезы об экспоненциальном распределении времени между послылками использовался критерий хи-квадрат. Для проверки гипотезы необходимо: область значений случайной величины, в зависимости от размера, поделить на k равных интервалов, затем подсчитать значения o_j - количество попаданий выборки размера n в каждый интервал. Далее вычислить теоретическую вероятность p_j попасть в j -тый интервал. Статистика критерия хи-квадрат вычисляется по формуле 2.3.

$$\hat{\chi}^2 = \sum_{i=1}^k \frac{(o_j - np_j)^2}{np_j} \quad (2.3)$$

Для того, чтобы отвергнуть гипотезу о распределении, статистика критерия должны быть больше или равной критической точке, равной квантили случайной величины с распределением хи-квадрат с $k - 1$ степенями свободы, на уровне значимости α (формула 2.4).

$$\hat{\chi}^2 \geq \chi_{1-\alpha, k-1}^2 \quad (2.4)$$

В таблице 2.1 представлены гипотезы, не противоречащие опытным данным с уровнем значимости 0,1:

Чемпионат ПетрГУ 2023		Школьный этап 2023	
Hour	λ	Hour	λ
1	1 / 30	20 min	1 / 10
2-3	1 / 130	1-3	1 / 20
4-5	1 / 70		
Чемпионат ПетрГУ 2019		Школьный этап 2022	
Hour	λ	Hour	λ
1-2	1 / 25	1-3	1 / 20
3-4	1 / 30		
5	1 / 15		

Таблица 2.1: Интенсивности

В таблице 2.1 указано соответствие между отрезком времени контеста и

λ — параметром распределения посылок, средним количеством посылок в единицу времени (секунду) на отрезке. Видно, что самая большая интенсивность — это $\frac{1}{10}$, продолжительность отрезка времени, к которому она относится — 20 минут. Наибольшая интенсивность была достигнута на школьном этапе ВСОШ 2023 в Петрозаводске, в котором было 388 участников. Можно сделать вывод о том, что подобные показатели характеризуют наиболее напряжённую часть соревнования с большим количеством участников. Средняя интенсивность для всех соревнований — $\frac{1}{30}$.

2.2 Система массового обслуживания

2.2.1 Определения

«Система массового обслуживания (СМО) — система, предназначенная для многократно повторяющегося (многоразового) использования при решении однотипных задач.

Процесс обслуживания — решение такой системой возникающей типовой задачи.

Задача обслуживания — повторяющаяся типовая задача, возникающая в СМО.

Заявка (требование) — объект обслуживания в СМО, для которого решается типовая задача обслуживания.

Поток заявок — последовательность объектов, подлежащих обслуживанию в СМО.

Канал обслуживания — обслуживающая единица, т. е. элемент системы, предназначенный для решения задачи обслуживания.

Очередь — последовательность заявок, ожидающих обслуживания.

Дисциплина обслуживания — правила, определяющие порядок выбора заявок из числа поступивших и порядок их распределения между каналами обслуживания.

Показатели эффективности СМО — показатели, описывающие способность СМО справляться с потоком заявок.» [1]

В проектируемой системе заявкой будет считаться посылка участника соревнования, обслуживанием - тестирование посылки, в качестве показателей эффективности СМО были выбраны среднее и максимальное время пребывания посылки в системе до завершения ее обслуживания.

2.2.2 Тип системы

Система массового обслуживания, которая наиболее подходит для моделирования разрабатываемого модуля, в нотации Кэнделла записывается следующим образом:

$$M \backslash G \backslash n \backslash \infty \backslash FIFO$$

- М — входной поток является Пуассоновским;
- G — распределение времени обслуживания посылок не указано;
- n — количество исполнителей;
- ∞ — очередь неограничена;
- FIFO (First in, first out) — очередь упорядочена в порядке поступления посылок на сервер.

2.3 Расчет оценочных значений

Для дальнейшей оптимизации системы рассчитаем оценки среднего времени обработки посылки для фиксированного числа исполнителей n . В качестве интенсивности потока обслуживания μ взято оценочное среднее $\frac{1}{30}$. Для расчета использовались формулы, использующие предельные вероятности схемы процесса гибели и размножения для вычисления среднего времени обработки. Пусть λ — интенсивность входного потока, μ — интенсивность потока обслуживания, n — количество исполнителей. Обозначим $\rho = \frac{\lambda}{\mu}$. Если $\frac{\rho}{n} < 1$, то предельных вероятностей не существует, система не будет справляться с потоком заявок. Иначе, по формуле 2.5 можно вычислить предельную вероятность того, что

очередь заявок на обслуживание будет пустой.

$$p_0 = \left(1 + \rho + \frac{\rho^2}{2!} + \frac{\rho^3}{3!} + \dots + \frac{\rho^n}{n!} + \frac{\rho^{n+1}}{n!(n - \rho)}\right)^{-1} \quad (2.5)$$

Среднее число заявок в очереди вычисляется по формуле 2.6, среднее число заявок под обслуживанием вычисляется по формуле 2.7.

$$L_{queue} = p_0 \frac{n\rho^{n+1}}{n!(n - \rho)^2} \quad (2.6)$$

$$L_{process} = \rho \quad (2.7)$$

Среднее время пребывания заявки в системе вычисляется по формуле 2.8.

$$T = \frac{L_{queue} + L_{process}}{\lambda} \quad (2.8)$$

λ	n = 3	n = 4	n = 5	n = 6
1 / 10	∞	45 сек	33,5 сек	31 сек
1 / 20	34,7 сек	30,8 сек	30,1 сек	30,03 сек
1 / 70	30,1 сек	30,01 сек	30,00 сек	30,00 сек

Таблица 2.2: Оценочное время пребывания в системе

В таблице 2.2 указано среднее оценочное время обслуживания при фиксированном числе исполнителей и фиксированной средней интенсивности. Видно, что для каждой интенсивности существует оптимальное число исполнителей, увеличение которого дает незначительное уменьшение.

2.4 Специализация исполнителей

Рассмотрим несколько потоков посылок в тестирующей системе:

1. Поток посылок во время **соревнования** (интенсивность: $\frac{1}{10} - \frac{1}{30}$) потребует самое большое количество исполнителей — 5;
2. Во время **дорешки** интенсивность будет достаточно мала — будем считать ее $\frac{1}{70}$, а приоритет обработки посылок — минимальный. Определим количество исполнителей — 3;

3. Посылки от **жюри** случаются крайне редко и имеют высочайший приоритет. Подобные посылки не требуют отдельных исполнителей, но займут первых свободных исполнителей;
4. Поток посылок, посланных на **перетестирование** не имеет интенсивности, т.к. они приходят группой и их необходимо протестировать сразу все. Приоритет должен быть выше чем у обычных посылок, но не занимать всех исполнителей в системе.

Исходя из этого можно рассчитать, сколько всего потребуется исполнителей. Также исполнителям невыгодно простаивать, даже если пришла посылка не их специализации, поэтому для каждого исполнителя составим приоритет каждого потока посылок. На рис. 2.1 изображен приоритет каждого потока посылок для каждого из исполнителей (сверху — наибольший, снизу — наименьший). В проектируемой системе стоит оставить возможность модифицировать эту схему, т. к. исполнителей может быть меньше чем 8.

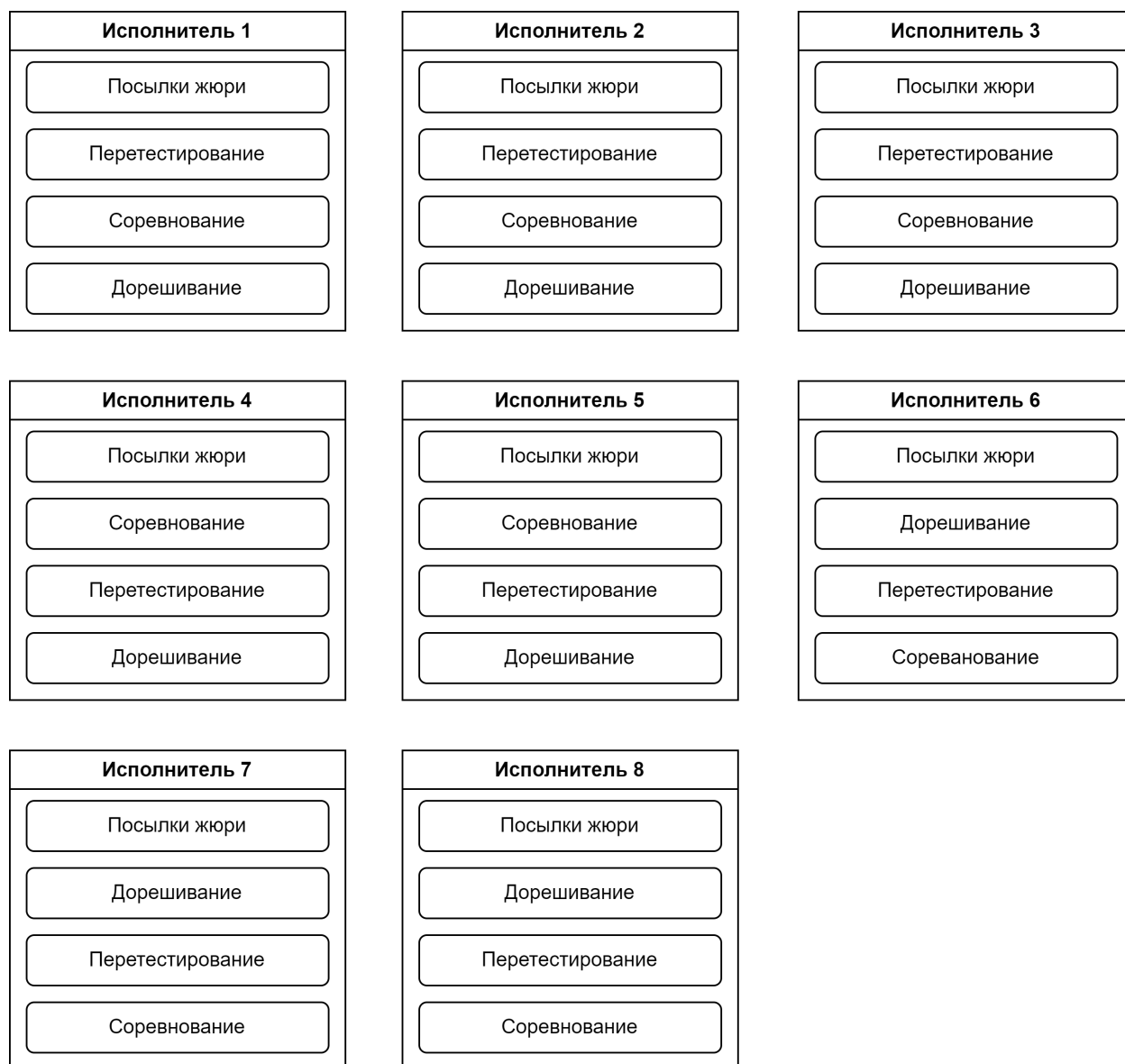


Рис. 2.1: Спецификация исполнителей

Глава 3

Алгоритм управления очередью

3.1 Целевая функция

В качестве целевой функции возьмем $f(M) = E[M] \rightarrow \min$, где M — случайная величина, описывающая максимальное время ожидания обработки посылки за отрезок времени с фиксированной интенсивностью. Для фиксированного $f(M)$ необходимо минимизировать среднее время ожидания $E[X]$, где X — время ожидания обработки одной посылки.

3.2 Разработка алгоритмов

3.2.1 FifoQueue

FifoQueue — базовый алгоритм управления очередью, реализовывающий принцип FIFO (first in, first out), то есть посылки в нем обрабатываются в порядке их поступления в очередь. В предыдущей главе для этого алгоритма вычислялись оценочные значения. Одна посылка отправляется на одного исполнителя, на котором посылка проходит все тесты. Если исполнитель освобождается и очередь не пустая, то посылка из очереди немедленно отправляется на исполнителя, чтобы он не простаивал. Для моделирования будем считать, что каждая посылка обрабатывается константное среднее время, выбранное для каждой задачи. Далее будем называть это время оценкой времени обработки.

3.2.2 OnlyAskQueue

Алгоритм OnlyAskQueue проталкивает вперед послыки с наименьшей оценкой времени обработки. Несложно показать, что подобный алгоритм уменьшает среднее время ожидания, однако он может привести к старвингу — блокировке послыки с большим временем обработки. Целью моделирования было определение степени оптимизации среднего времени ожидания, и увеличения среднего максимального времени.

3.2.3 MaxFirstQueue

Попробуем сосредоточиться на оптимизации среднего максимального времени ожидания. В момент m , когда необходимо отправить задачу на обработку, оценим время ожидания каждой послыки i в очереди как $(m - a_i) + t_i + \sum_{j=0}^{i-1} t_j$, где t_i — оценочное время обработки i -той послыки, a_i — момент, когда i -тая послыка пришла в очередь. Найдём послыку с максимальной оценкой и попробуем протолкнуть её вперед, с целью уменьшить максимальное время ожидания. Однако, можно показать, что это, в лучшем случае, не изменит $f(M)$. Допустим мы выбрали послыку i . Её оценка равна $s_i = (m - a_i) + t_i + \sum_{j=0}^{i-1} t_j$. Рассмотрим послыку с номером $i - 1$, её оценка с учетом перемещения послыки i в начало очереди равна: $s'_{i-1} = (m - a_{i-1}) + t_{i-1} + \sum_{j=0}^{i-2} t_j + t_i$.

$s'_{i-1} \geq s_i$, следовательно, максимальное время ожидания может только увеличиться.

Однако, в данном рассуждении никак не учтен тот факт, что в системе более одного исполнителя. Можно поробовать выделить отдельного исполнителя для обработки «тяжелых» послылок. При моделировании алгоритма MaxFirstQueue, послылки с наибольшим оценочным временем ожидания проталкиваются вперед, но только, если они попадут на исполнителя с номером 0.

3.2.4 TestByTestQueue

Особенностью потока послылок, отличающей его от типичного потока послылок в системах массового обслуживания, является то, что решение

участника необходимо протестировать на некотором количестве тестов, так что можно представить посылку как множество посылок, которые необходимо запустить один раз на соответствующем тесте и которые приходят единовременно. Алгоритм `TestByTestQueue` запускает посылку независимо на разных исполнителях на разных тестах. Также, если программа выдаст неправильный ответ на какой-то тест, то необходимо завершить тестирование этой посылки.

Оценкой времени обработки для каждого теста будем считать максимальное ограничение по времени для этого теста.

3.2.5 `SortTestQueue`

Опыт участия в соревнованиях показывает, что довольно часто в задаче есть несколько «краеугольных» тестов, которые не проходят большинство посылок. Если для каждого теста поддерживать количество посылок, которые не прошли его, то можно, в рамках задачи, отсортировать тесты по вероятности, с которой решения не пройдут тест. Это потребует проведения соревнования по особым правилам, так как не будет возможности сообщать вердикты вида «*x* — первый тест, на котором решение отработало некорректно». Единственное, что можно гарантировать, это то, что для задачи существует тест, на котором был получен соответствующий вердикт.

3.3 Имитационная модель

Для моделирования и сравнения разработанных алгоритмов, на языке программирования Python была создана имитационная модель процесса соревнования, потока посылок и их обработки.

Архитектура имитационной модели:

1. *parsed_data* — данные, необходимые для моделирования потока посылок.
 - (a) *parsed* — данные о времени посылок участников в этапе ВСОШ 2023.

- (b) *parsed_2019* — данные о времени посылок участников в этапе Чемпионата ПетрГУ 2019 года.
- (c) *parsed_2023* — данные о времени посылок участников в этапе Чемпионата ПетрГУ 2023 года.
- (d) *parsed_ptz* — данные о задаче и тесте, где посылка отработала некорректно в контексте с Петрозаводских тренировочных сборов по программированию.
- (e) *parsed_s22* — данные о времени посылок участников в этапе ВСОШ 2022.

2. *compare.py*

- (a) *compareQueues(queue1, queue2, genEvents)* — построение графиков среднего времени ожидания и максимального времени ожидания для разных интенсивностей для двух различных алгоритмах управления очередью — *queue1* и *queue2*. Для генерации событий используется функция *genEvents*.

3. *events.py*

- (a) *Event()* — класс хранящий информацию о посылке, а именно: время посылки, количество тестов, ограничение по времени на тест, тест, на котором посылка отработала некорректно, номер задачи.

4. *fifo.py*

- (a) *FifoQueue(Queue)* — класс, реализующий алгоритм управления очередью *FifoQueue*. Наследуется от класса *Queue*.

5. *main.py*

- (a) *genEvents1(ev_in_sec)* — генератор посылок с случайным распределением времени, экспоненциально распределенных с параметром *ev_in_sec*. Оценочное время выполнения посылок взяты с этапа ВСОШ 2023.

- (b) *genEvents2(ev_in_sec)* — генератор посылок, где время посылок взято из *parsed*. Оценочное время выполнения посылок взяты с этапа ВСОШ 2023.
- (c) *genEvents3(ev_in_sec)* — генератор посылок с случайным распределением времени, экспоненциально распределенных с параметром *ev_in_sec*. Оценочное время выполнения посылок взяты из контекста с Петрозаводских тренировочных сборов, также из файла *parsed_ptz* взята информация о том, какая задача отправлялась и на каком тесте она не отрабатывает корректно.

6. *maxfirst.py*

- (a) *MaxFirstQueue(Queue)* — класс, реализующий алгоритм управления очередью *MaxFirstQueue*. Наследуется от класса *Queue*. Перегружает метод *newTaskForJudge*.

7. *model.py*

- (a) *launchModel(events, queue)* — функция, моделирующая процесс соревнования и управляющая очередью с помощью алгоритма *queue*.

8. *onlyask.py*

- (a) *OnlyAskQueue(Queue)* — класс, реализующий алгоритм управления очередью *OnlyAskQueue*. Наследуется от класса *Queue*. Перегружает метод *newTaskForJudge*.

9. *parser.py* — вспомогательная программа для парсинга.

10. *queue.py*

- (a) *Queue()* — класс, реализующий общие для алгоритмов управления очередями функции.
 - i. *tact(moment)* — моделирование всех событий, которые произошли в очереди за момент времени *moment*.

- ii. *freeJudge(moment, judge)* — завершение обработки на исполнителе номер *judge*.
- iii. *newTaskForJudge(judge, moment)* — получить слудующую посылку для тестирования на исполнителя номер *judge*, если очередь не пуста.
- iv. *push(event)* — добавление посылки в очередь.
- v. *printStatistic()* — вывод среднего и максимального времени ожидания.
- vi. *getMax()* — получение максимального времени ожидания.
- vii. *getAverage()* — получение среднего времени ожидания.

11. *sorttest.py*

- (a) *SortTestQueue(TestByTestQueue)* — класс, реализующий алгоритм управления очередью *SortTestQueue*. Наследуется от класса *TestByTestQueue*. Перегружает методы *freeJudge*, *push*.

12. *testbytest.py*

- (a) *TestByTestQueue(Queue)* — класс, реализующий алгоритм управления очередью *TestByTestQueue*. Наследуется от класса *Queue*. Перегружает методы *newTaskForJudge*, *freeJudge*, *push*.

Ссылка на реализацию имитационной модели:

<https://github.com/bugwithbutts/smo>.

3.4 Анализ алгоритмов

3.4.1 OnlyAskQueue

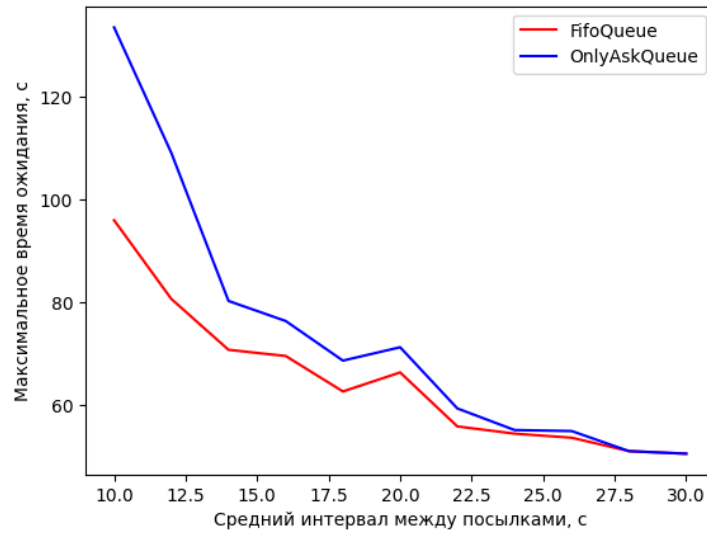


Рис. 3.1: Максимальное время ожидания

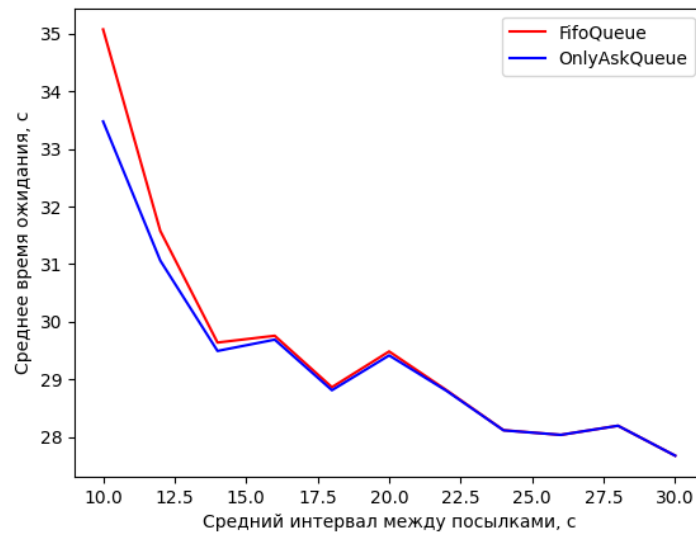


Рис. 3.2: Среднее время ожидания

На рисунках 3.1 и 3.2 заметно, что при использовании OnlyAskQueue, максимальное время ожидания возрастает больше, чем уменьшается среднее время ожидания.

3.4.2 MaxFirstQueue

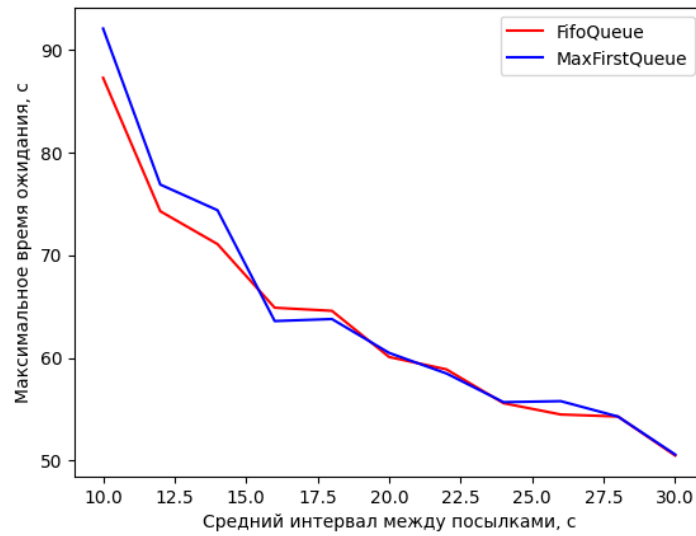


Рис. 3.3: Максимальное время ожидания

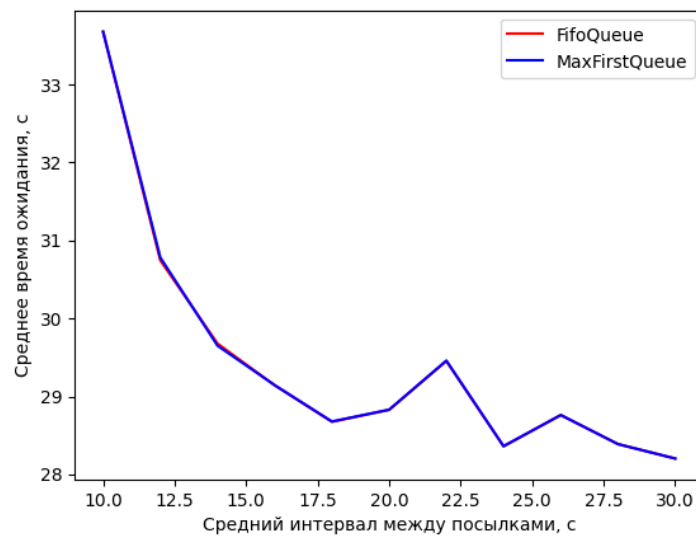


Рис. 3.4: Среднее время ожидания

На рисунке 3.3 видно, что при использовании MaxFirstQueue, максимальное время ожидания зачастую становится больше. На рисунке 3.4 видно, что среднее время ожидания остается практически неизменным.

3.4.3 TestByTestQueue

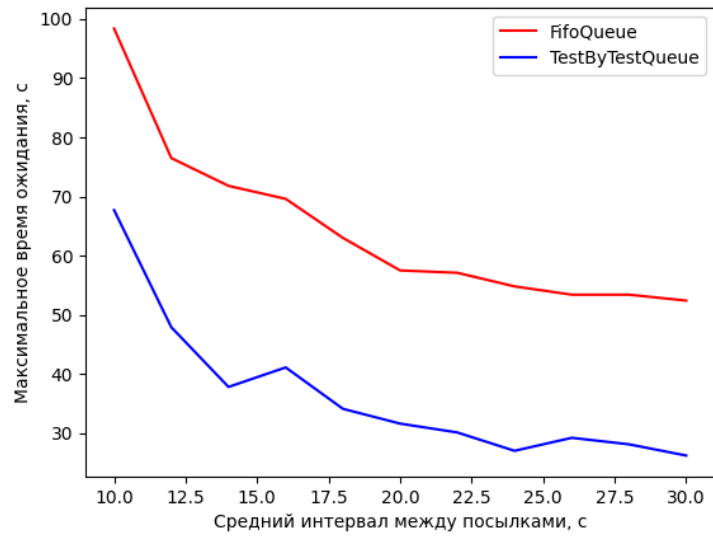


Рис. 3.5: Максимальное время ожидания

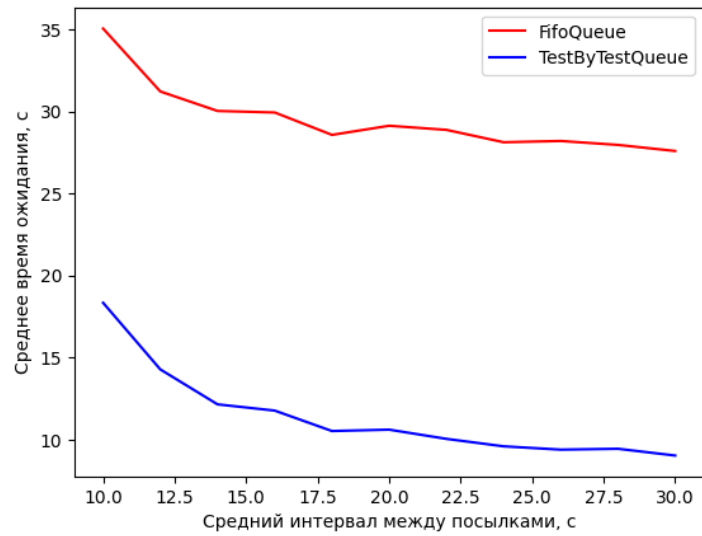


Рис. 3.6: Среднее время ожидания

На рисунках 3.5 и 3.6 видно, что параллелизация тестов дает существенную оптимизацию к $f(M)$.

3.4.4 SortTestQueue

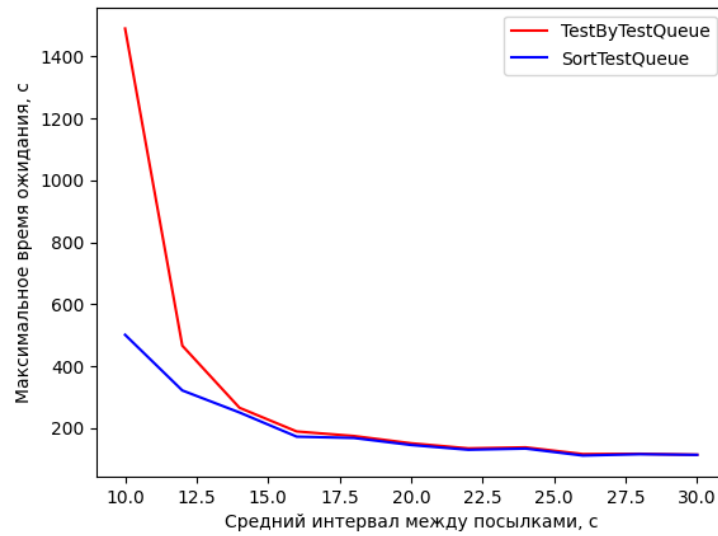


Рис. 3.7: Максимальное время ожидания

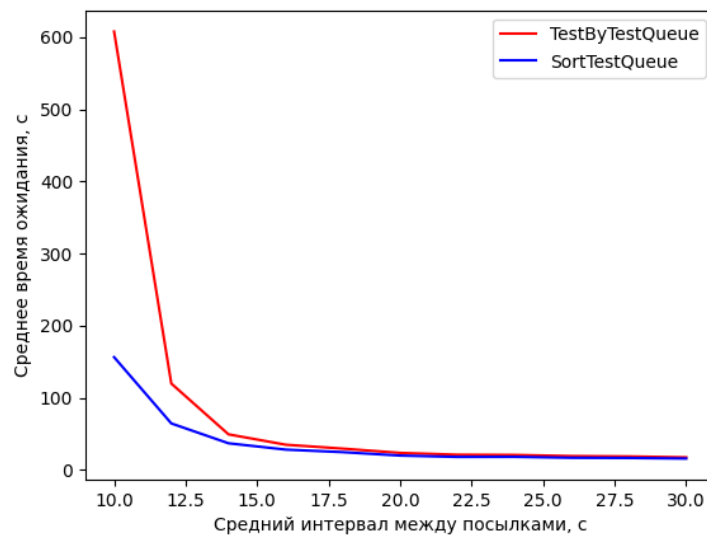


Рис. 3.8: Среднее время ожидания

На рисунках 3.7 и 3.8 видно, что и среднее и максимальное время ожидания уменьшаются при использовании алгоритма SortTestQueue, но только при достаточно больших интенсивностях, иначе изменения незначительны.

3.5 Компиляция

При потестовом тестировании возникает проблема с компиляцией — чтобы запустить одну программу на нескольких разных исполнителях, то ее нужно несколько раз скомпилировать на каждом. Однако, в проектируемой системе планируется использование многоядерных процессоров, и на одном сервере может находиться несколько исполнителей и они могут использовать единожды скомпилированный исполняемый файл. Поэтому распараллеливание по тестам будет происходить только в рамках одного компьютера.

Процесс компиляции задействует только одно ядро, другие же простаивают. Для оптимизации, можно начинать компиляцию до того, как тестирование предыдущей задачи на исполнителе завершилось.

3.6 Итоговый алгоритм

Выводы:

- Проведенное исследование показывает, что изменение порядка посылок в очереди приводит к увеличению значения $f(M)$;
- Потестовое тестирование крайне выгодно, однако необходимо учесть проблему с компиляцией, описанную выше;
- Сортировка тестов в рамках одной задачи имеет смысл только при большой интенсивности посылок, так что ее стоит сделать опциональной для соревнования.

Назовем исполнителем определенное количество ядер, где одно ядро отвечает за компиляцию приходящих посылок, а другие — за тестирование. На одном компьютере можно создать несколько исполнителей, если на нем достаточно ядер. Для исполнителя должна быть определена спецификация, как на рисунке 2.1.

Планировщик, запущенный на основном сервере, получает посылку на тестирование. После этого, если для конкурса стоит опция сортировки тестов, тесты сортируются по вероятности, с которой посылка их

не пройдет. Затем, в зависимости от типа посылки (соревнование, дорешивание, посылка жюри, перетестирование), она добавляется в одну из очередей на обработку.

Исполнитель компилирует посылки на отдельном ядре, а затем помещает их в очередь на исполнение, если там нет тестирующейся задачи, иначе ждет освобождения очереди. Посылка в очереди на исполнение потестово запускается на ядрах, предназначенных для тестирования, до тех пор, пока не будут пройдены все тесты или решение отработает некорректно на каком-нибудь из них.

Если ядро для компиляции не занято компиляцией и не ждет освобождения очереди на тестирование, исполнитель запрашивает новую задачу у планировщика.

Глава 4

Разработка программного модуля

4.1 Проектирование

Submission — класс, описывающий посылку в систему. Содержит следующие поля:

1. *id* — уникальный идентификатор посылки;
2. *tl* — ограничение по времени для одного теста;
3. *ml* — ограничение по памяти для одного теста;
4. *fails* — количество раз, которое решения участников не прошли данный тест;
5. *taskId* — уникальный идентификатор задачи;
6. *contestId* — уникальный идентификатор конкурса;
7. *code* — исходный код решения;
8. *compile* — компилятор, которые необходимо использовать для посылки;
9. *verdict* — динамическое поле, которое в дальнейшем заполняется вердиктом на посылку;
10. *pass* — динамическое поле, содержащее последний тест, который прошло решение.

Планировщик *Scheduler* состоит из нескольких очередей:

1. *ContestQueue* — очередь для посылок с соревнования;
2. *UpsolvingQueue* — очередь для посылок из дорешивания;
3. *RetestQueue* — очередь для посылок, отправленных на перетестирование;
4. *JuryQueue* — очередь для посылок, сделанных жюри соревнования.

Также он должен содержать следующие функции:

1. *submissionHandler()* — функция, проверяющая нет ли новых посылок в систему. Посылки приходят в виде json, который содержит информацию о посылке. Если указана соответствующая опция, тесты в посылке сортируются по вероятности, с которой решение их не пройдет;
2. *getInfoFromJudges()* — функция, получающая информацию от исполнителей, такую как: необходимо ли отправить задачу на исполнителя, какая задача сейчас тестируется и на каком тесте;
3. *sendSubmission(judge, submission)* — функция, которая отправляет посылку *submission* на соответствующий *judge*.

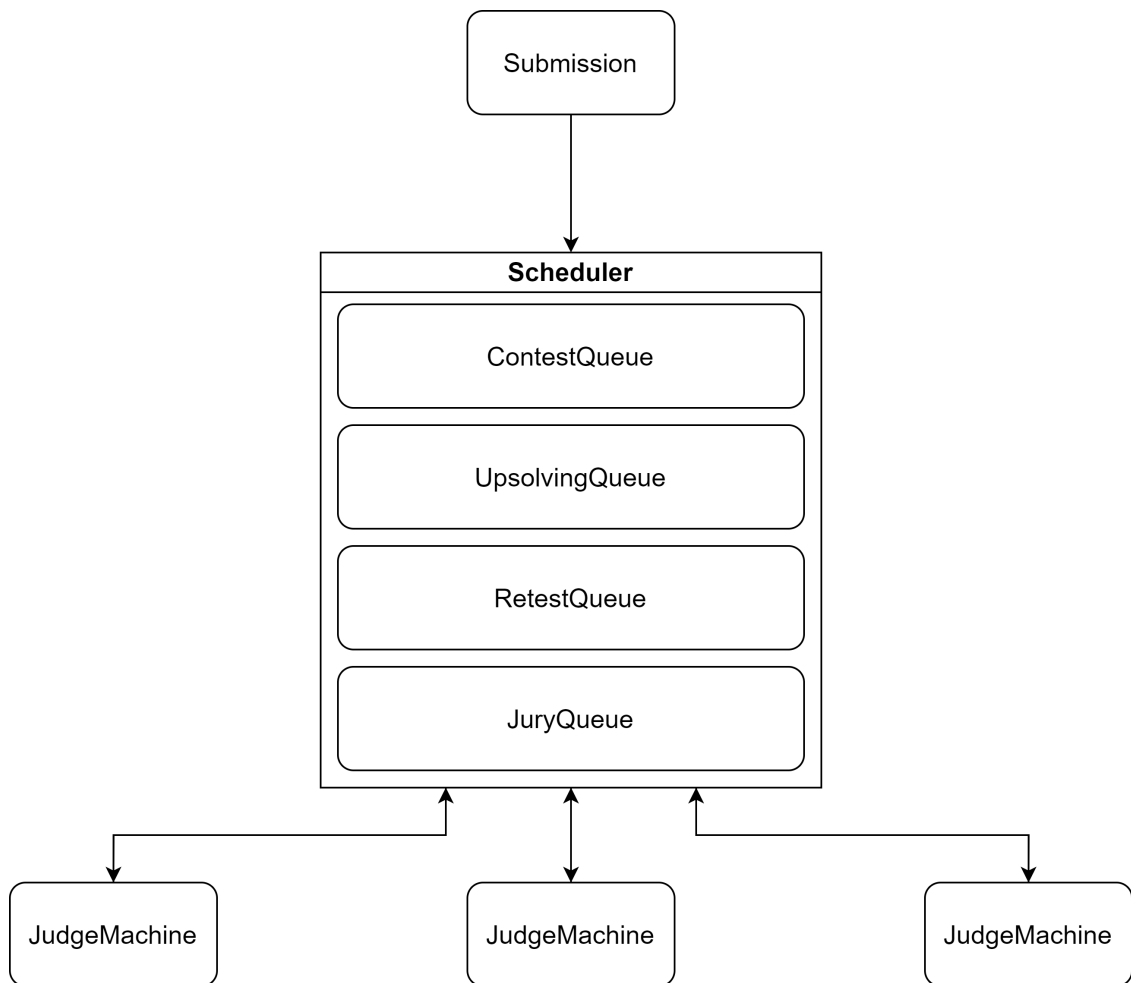


Рис. 4.1: Устройство планировщика

Планировщик отправляет послыки не напрямую в исполнителя, а в модуль *JudgeMachine*, который запущен на сервере, на котором будет происходить тестирование. *JudgeMachine* пересылает приходящие послыки на исполнителя (*Judge*), который будет тестировать посылку. В модуле задается конфигурация исполнителей (сколько ядер в каждом). *JudgeMachine* содержит следующие функции:

1. *submissionHandler()* — функция, которая проверяет, не отправляется ли новая посылка на этот компьютер;
2. *newSubmission(submission, judge)* — функция, которая отправляет посылку на исполнителя *judge*;
3. *reportSubmission()* — функция, которая отправляет данные о посылках планировщику. Информацию берет из объектов *submission*, которые передаются в исполнителей по ссылкам.

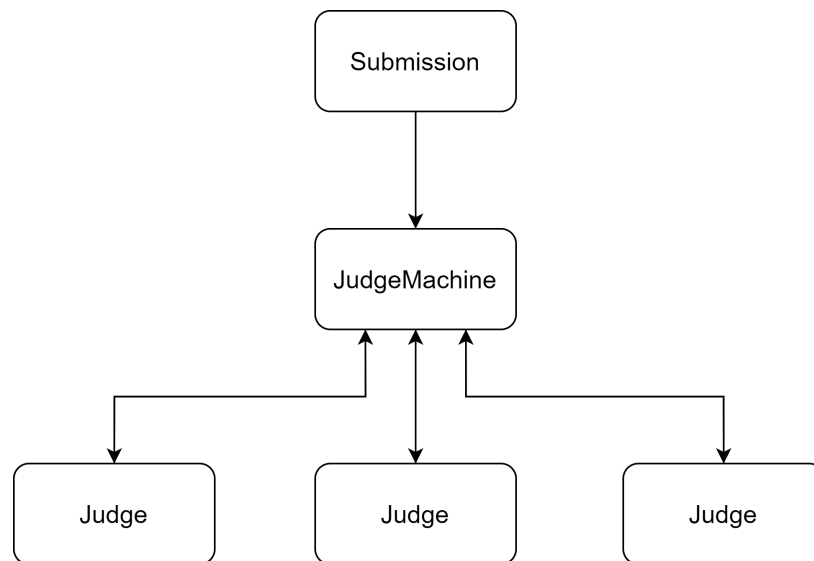


Рис. 4.2: Устройство компьютера для тестирования

Judge — класс, описывающий исполнителей. Содержит следующие функции:

1. *compile(submission)* — запуск процесса компиляции *submission* на ядре для компиляции и дальнейшее добавление в очередь на исполнение (*queueToTest*). Если очередь не пуста, идет блокировка процесса;

2. *test(submission)* — тестирование посылки, находящейся в очереди;
3. *testOnCore(submission, core, test)* — запуск решения *submission* на ядре *core* с подачей на ввод теста *test*;

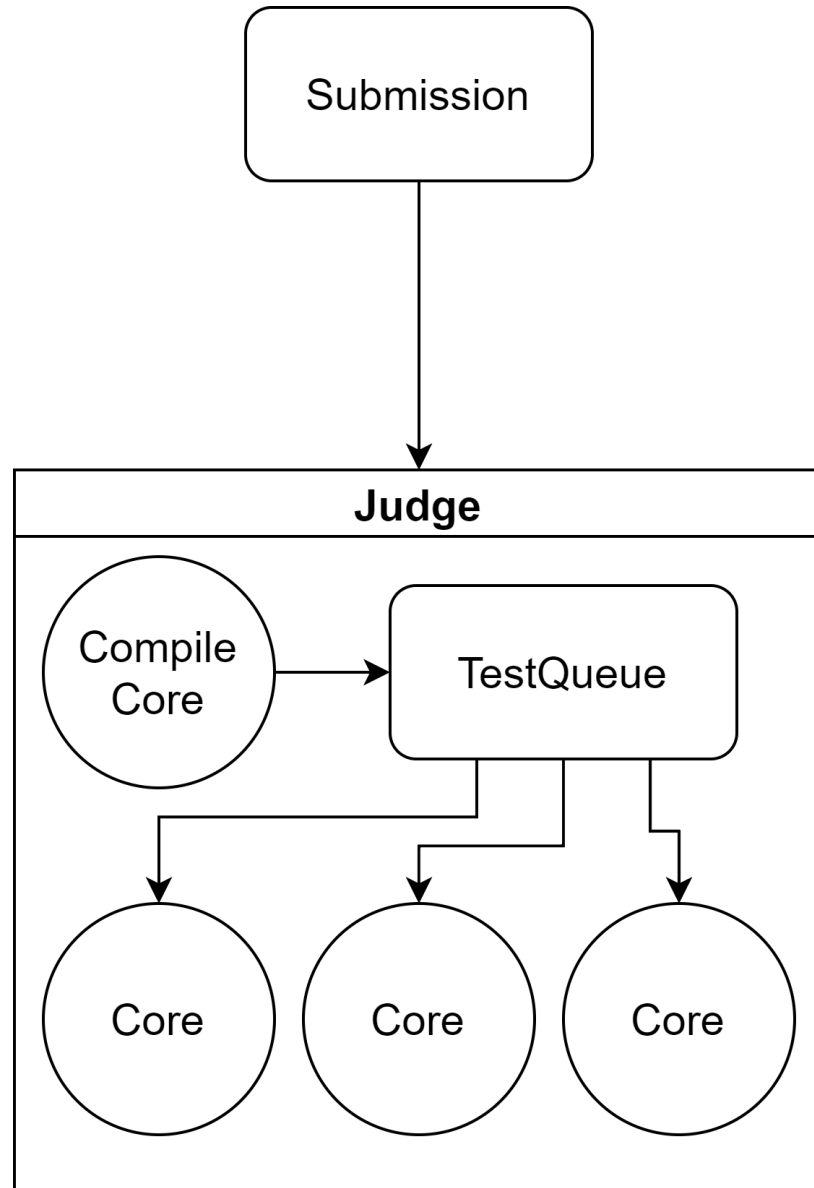


Рис. 4.3: Устройство исполнителя

4.2 Разработка

Разработка модуля велась на языке программирования Python. Для обмена сообщениями между подсистемами, находящимися на разных компьютерах, а также для связи с остальной системой, использовались

веб-сокеты и unix-сокеты. Для управления ядрами компьютера использовалась библиотека `psutil`, для создания потоков внутри модуля — библиотеки `threading` и `multiprocessing`. Для работы с `json` использовалась библиотека `json`.

Стоит отметить, что за доступ исполнителей к тестам задачи отвечает другой программный модуль, называемый балансировщиком.

Ссылка на реализацию программного модуля:
<https://github.com/bugwithbutts/judges>.

Заключение

В ходе выполнения работы были достигнуты следующие результаты:

1. Рассмотрены и проанализированы аналоги проектируемой системы;
2. Проверены гипотезы о Пуассоновском потоке посылок во время контеста;
3. С помощью теории массового обслуживания вычислены оценочные значения среднего времени ожидания для посылок в очереди;
4. Разработана имитационная модель для проверки и сравнения алгоритмов управления очередью;
5. Разработан итоговый алгоритм для программного модуля;
6. Спроектирован и разработан программный модуль для управления очередью посылок.

В дальнейшем планируется внедрение программного модуля в тестирующую систему, а также модификация его для других типов задач.

Библиографический список использованной литературы

1. Плескунов, Михаил Александрович. Теория массового обслуживания : учебное пособие / М. А. Плескунов ; М-во науки и высшего образования РФ, Урал. федер. ун-т. — Екатеринбург : Изд-во Урал. ун-та, 2022. — 264 с.
2. Бородина, Александра Валентиновна. Статистические критерии в анализе данных : учебное пособие / А. В. Бородина, Р. С. Некрасова ; М-во науки и высш. образования Рос. Федерации, Федер. гос. бюджет. образоват. учреждение высш. образования Петрозав. гос. ун-т. — Петрозаводск : Издательство ПетрГУ, 2023. — 45 с.
3. Процесс Пуассона [Электронный ресурс] URL: https://ru.wikipedia.org/wiki/Процесс_Пуассона.
4. Документация системы ejudge [Электронный ресурс] URL: <https://ejudge.ru/wiki>.
5. Изучаем Latency: теория массового обслуживания [Электронный ресурс] URL: <https://habr.com/ru/companies/yandex/articles/431650/>
6. Приказ Министерства спорта Российской Федерации № 584 от 11.08.23 «Об утверждении правил вида спорта «спортивное программирование»»